3. Use an abstract factory pattern to realize the computer configuration store. The requirements are as follows:

（1） The computer comprises a CPU, motherboard, graphics card, memory, hard disk, and other accessories. There are two options for computer configuration.

（2） According to the configuration plan, specific configuration information and computer prices can be displayed.

（3） Write a test class to use the ComputerFactory to generate two different configurations of computers and print their configuration and price.

ComputerFactory

```
package experiment5_3;


interface ComputerFactory {
    CPU createCPU();
    GraphicsCard createGraphicsCard();
    HardDisk createHardDisk();
    Memory createMemory();
    Motherboard createMotherboard();
}
```

CPU

```
package experiment5_3;


interface CPU {
    void produceCPU();
    double getPrice();
}
```

GraphicsCard

```
package experiment5_3;


interface GraphicsCard {
    void produceGraphicsCard();
    double getPrice();
}
```

HardDisk

```
package experiment5_3;


interface HardDisk {
    void produceHardDisk();
    double getPrice();
}
```

Memory

```
package experiment5_3;


interface Memory {
    void produceMemory();
    double getPrice();
}
```

Motherboard

```
package experiment5_3;


interface Motherboard {
    void produceMotherboard();
    double getPrice();
}
```

ProComputerFactory

```
package experiment5_3;


public class ProComputerFactory implements
ComputerFactory{
    @Override
    public CPU createCPU() {
        return new ProCPU();
    }


    @Override
    public GraphicsCard createGraphicsCard() {
        return new ProGraphicsCard();
    }


    @Override
```

```
    public HardDisk createHardDisk() {
        return new ProHardDisk();
    }

    @Override
    public Memory createMemory() {
        return new ProMemory();
    }

    @Override
    public Motherboard createMotherboard() {
        return new ProMotherboard();
    }

}
```

ProCPU

```
package experiment5_3;

public class ProCPU implements CPU{
    @Override
    public void produceCPU() {
        System.out.print("CPU: Pro CPU");
```

```
    }

    @Override
    public double getPrice() {
        return 3000;
    }
}
```

ProGraphicsCard

```
package experiment5_3;

public class ProGraphicsCard implements GraphicsCard{
    @Override
    public void produceGraphicsCard() {
        System.out.print("Graphics Card: Pro Graphics Card");
    }

    @Override
    public double getPrice() {
        return 1200;
    }
}
```

ProHardDisk

```java
package experiment5_3;

public class ProHardDisk implements HardDisk{
    @Override
    public void produceHardDisk() {
        System.out.print("Hard Disk: Pro Hard Disk");
    }

    @Override
    public double getPrice() {
        return 800;
    }
}
```

ProMemory

```java
package experiment5_3;

public class ProMemory implements Memory{
    @Override
    public void produceMemory() {
        System.out.print("Memory: Pro Memory");
    }
```

```
    @Override
    public double getPrice() {
        return 1000;
    }
}
```

ProMotherboard

```
package experiments5_3;

public class ProMotherboard implements Motherboard{
    @Override
    public void produceMotherboard() {
        System.out.print("Motherboard: Pro Motherboard");
    }

    @Override
    public double getPrice() {
        return 1100;
    }
}
```

StandardComputerFactory

```
package experiments5_3;
```

```
public class StandardComputerFactory implements
ComputerFactory{
    @Override
    public CPU createCPU() {
        return new StandardCPU();
    }

    @Override
    public GraphicsCard createGraphicsCard() {
        return new StandardGraphicsCard();
    }

    @Override
    public HardDisk createHardDisk() {
        return new StandardHardDisk();
    }

    @Override
    public Memory createMemory() {
        return new StandardMemory();
    }
```

```
    @Override
    public Motherboard createMotherboard() {
        return new StandardMotherboard();
    }
}
```

StandardCPU

```
package experiments5_3;

public class StandardCPU implements CPU{
    @Override
    public void produceCPU() {
        System.out.print("CPU: Standard CPU");
    }

    @Override
    public double getPrice() {
        return 2000;
    }
}
```

StandardGraphicsCard

```
package experiments5_3;
```

```
public class StandardGraphicsCard implements
GraphicsCard{
    @Override
    public void produceGraphicsCard() {
        System.out.print("GraphicsCard: Standard Graphics
Card");
    }

    @Override
    public double getPrice() {
        return 900;
    }
}
```

StandardHardDisk

```
package experiment5_3;

public class StandardHardDisk implements HardDisk{
    @Override
    public void produceHardDisk() {
        System.out.print("Hard Disk: Standard Hard Disk");
    }
```

```
    @Override
    public double getPrice() {
        return 800;
    }
}
```

StandardMemory

```
package experiments5_3;


public class StandardMemory implements Memory{
    @Override
    public void produceMemory() {
        System.out.print("Memory: Standard Memory");
    }

    @Override
    public double getPrice() {
        return 600;
    }
}
```

StandardMotherboard

```
package experiments5_3;
```

```
public class StandardMotherboard implements Motherboard{
    @Override
    public void produceMotherboard() {
        System.out.print("Motherboard: Standard
Motherboard");
    }

    @Override
    public double getPrice() {
        return 700;
    }
}
```

Test

```
package experiment5_3;

public class Test {
    public static void main(String[] args) {
        ComputerFactory pro = new ProComputerFactory();
        ComputerFactory standard = new
StandardComputerFactory();
        printDetails(pro);
        System.out.println();
```

```
        printDetails(standard);
    }
    public static void printDetails(ComputerFactory
computerFactory){
        CPU cpu = computerFactory.createCPU();
        GraphicsCard graphicsCard =
computerFactory.createGraphicsCard();
        HardDisk hardDisk =
computerFactory.createHardDisk();
        Memory memory = computerFactory.createMemory();
        Motherboard motherboard =
computerFactory.createMotherboard();

        cpu.produceCPU();
        System.out.print(" => The price of CPU is:
"+cpu.getPrice()+"\n");
        graphicsCard.produceGraphicsCard();
        System.out.print(" => The price of graphics card is:
"+graphicsCard.getPrice()+"\n");
        hardDisk.produceHardDisk();
        System.out.print(" => The price of hard Disk is:
"+hardDisk.getPrice()+"\n");
```

```
        memory.produceMemory();
        System.out.print(" => The price of memory is:
"+memory.getPrice()+"\n");
        motherboard.produceMotherboard();
        System.out.print(" => The price of motherboard is:
"+motherboard.getPrice()+"\n");

        double totalPrice =
cpu.getPrice()+graphicsCard.getPrice()+hardDisk.getPrice()+
memory.getPrice()+motherboard.getPrice();
        System.out.println("The total price is "+totalPrice);
    }
}
```

Output

```
D:\JDK\bin\java.exe "-javaagent:D:\idea\IntelliJ IDEA 2021.2.1\lib\idea_rt.jar=569
CPU: Pro CPU => The price of CPU is: 3000.0
Graphics Card: Pro Graphics Card => The price of graphics card is: 1200.0
Hard Disk: Pro Hard Disk => The price of hard Disk is: 800.0
Memory: Pro Memory => The price of memory is: 1000.0
Motherboard: Pro Motherboard => The price of motherboard is: 1100.0
The total price is 7100.0

CPU: Standard CPU => The price of CPU is: 2000.0
GraphicsCard: Standard Graphics Card => The price of graphics card is: 900.0
Hard Disk: Standard Hard Disk => The price of hard Disk is: 800.0
Memory: Standard Memory => The price of memory is: 600.0
Motherboard: Standard Motherboard => The price of motherboard is: 700.0
The total price is 5000.0

Process finished with exit code 0
```